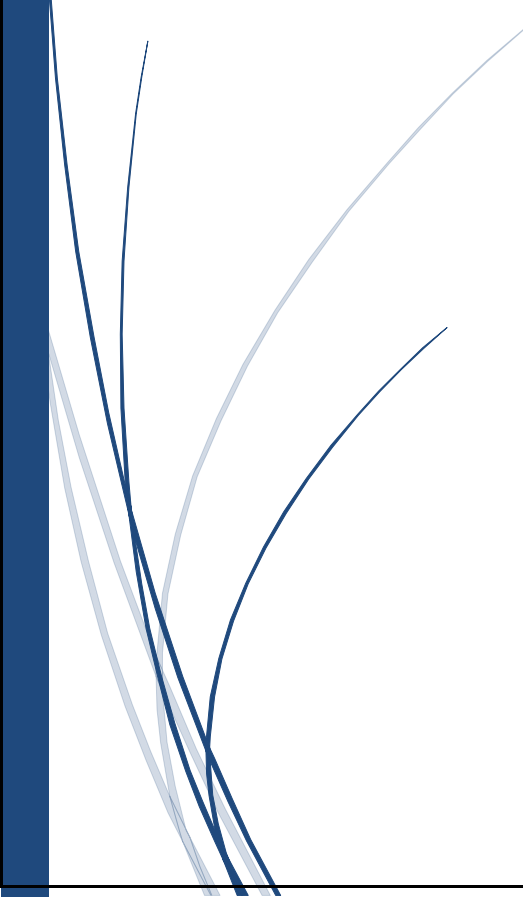




Fall Detection using Pose Estimation

using UR Fall Dataset, Le2i Dataset and
Montreal Dataset



Syeda Ayesha Wajahat

Fall Detection using Pose Estimation using UR Fall Dataset,
Le2i Dataset and Montr

Table of Contents

Fall Detection using Pose Estimation -UR Fall Dataset.....	4
1. Introduction	4
2. Methodology	4
2.1 Pose Estimation with MediaPipe.....	4
2.2 Feature Engineering.....	5
2.3 Temporal Sequence Processing.....	5
2.4 Model Architectures	5
3. Implementation Details.....	5
3.1 Tools & Libraries.....	5
3.2 Pipeline	6
4. Model-Specific Results and Analysis.....	6
4.1 Balanced Random Forest.....	6
4.2 Balanced Random Forest with SMOTE.....	8
4.3 LSTM	9
5. Conclusion:	10
6. Demo:.....	11
Fall Detection using Pose Estimation - Le2i Dataset.....	13
1. Dataset Structure Exploration	13
2. Keypoint Extraction using MediaPipe	14
3. Feature Extraction (Angles + Center of Gravity).....	14
4. Creating Temporal Sequences	14
5. Label Assignment.....	14
6. Model Building and Training.....	15
7. Evaluation Metrics.....	15
8. Animation/Demo of Fall.....	17
Conclusion	17
Fall Detection using Pose Estimation - Montreal Dataset.....	18
1. Introduction	18
2. Methodology	18
2.1 Pose Estimation with MediaPipe.....	18
2.2 Feature Engineering.....	18

2.3 Temporal Sequence Processing.....	19
2.4 Model Architectures	19
3. Implementation Details.....	19
3.1 Tools & Libraries.....	19
3.2 Pipeline	20
3.3 Demo Visualization.....	20
4. Model-Specific Results and Analysis.....	20
4.1 Random Forest (Baseline)	20
4.2 Balanced Random Forest.....	21
4.3 Random Forest with SMOTE.....	22
4.4 Balanced Random Forest with SMOTE.....	23
4.5 LSTM with Dropout (0.3)	24
4.6 LSTM without Dropout.....	25
5. Conclusion.....	27
6. Demo:	27
Summary:	29
1. Montreal Dataset:.....	29
Le2i Fall Detection Dataset:.....	29
UR Fall Detection Dataset:.....	29
Overall Observations:.....	29
Traditional Machine Learning:	29
Deep Learning (LSTM Models):.....	30
Class Imbalance:	30
Conclusion:	30
Demo links:.....	30

Fall Detection using Pose Estimation -UR Fall Dataset

1. Introduction

Problem Statement:

Detecting falls in real-time from video sequences is critical for elderly care, healthcare monitoring, and safety systems. This project aims to classify whether a person has fallen using pose estimation and temporal sequence modeling.

Dataset:

This dataset contains **70 (30 falls + 40 activities of daily living)** sequences:

- **Fall Scenarios** (Fall events are recorded with **2 Microsoft Kinect cameras** and corresponding **accelerometric data.**): Labeled 1.
- **Non-Fall Scenarios** (ADL events are recorded with only one device (camera 0) and accelerometer. Sensor data is collected using **PS Move (60Hz)** and **x-IMU (256Hz)** devices): Labeled 0.
- Since Camera 1 only has fall cases and we don't want to give our model extreme class imbalance, we don't have a single adl (non-fall) case for camera 1, assuming that our model may lead to biased predictions i.e predicting a fall at every camera 1 video in future; we decide to discard camera 1 videos. So now our dataset only consists of 30 fall and 40 non-fall cases

2. Methodology

2.1 Pose Estimation with MediaPipe

- **Keypoint Extraction:**
 - MediaPipe (v0.10.8) extracts **33 body keypoints** (x, y, z coordinates) per frame.
 - Zero-padding is applied if no pose is detected.
- **Output:**

- Keypoints and confidence scores (visibility) saved as .npy files.

2.2 Feature Engineering

- **Joint Angles:**

Angles at elbows, knees, shoulders, hips, neck, and torso computed using vector trigonometry.

- **Motion Dynamics:**

- Center of gravity (mean of all keypoints).
- Velocity and acceleration via finite differences.

- **Confidence Scores:** Visibility of each keypoint from MediaPipe.

2.3 Temporal Sequence Processing

- **Sliding Window:**

- Window size = 30 frames (~1 sec at 30 FPS).
- Step size = 10 frames.

- **Normalization:**

StandardScaler applied to ensure zero mean and unit variance.

2.4 Model Architectures

1. **Balanced Random Forest:** RF with class_weight='balanced' to handle class imbalance.
2. **Balanced RF with SMOTE:** Combined class weighting and SMOTE.
3. **LSTM without Dropout:** Simplified LSTM model without dropout layers.

3. Implementation Details

3.1 Tools & Libraries

- **Pose Estimation:** MediaPipe, OpenCV.

- **Feature Engineering:** NumPy, SciPy.
- **Machine Learning:** scikit-learn, imbalanced-learn (SMOTE).
- **Deep Learning:** TensorFlow/Keras.

3.2 Pipeline

1. Video Processing:

- Extract keypoints and confidence scores for each video.
- Save outputs to /keypoints and /confidence directories.

2. Feature Generation:

- Combine joint angles, motion dynamics, and confidence into flattened features.

3. Sequence Creation:

- Convert features into 30-frame sequences.

4. Training:

- 80-20 train-test split.
- Models saved to /models.

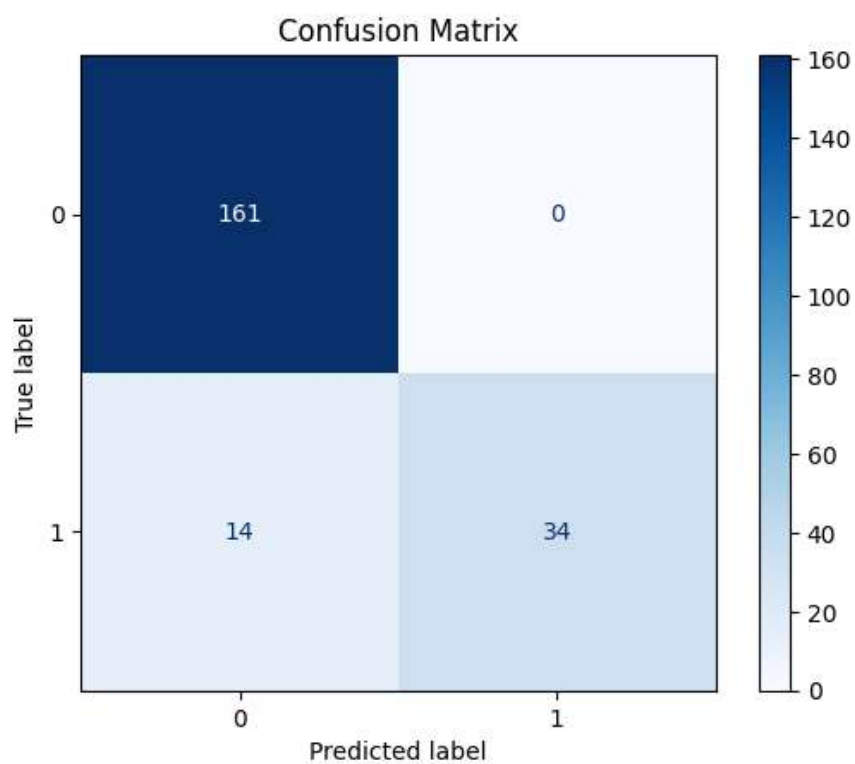
4. Model-Specific Results and Analysis

4.1 Balanced Random Forest

- **Configuration:** class_weight='balanced' to address class imbalance.
- **Key Metrics:**

Classification Report:

	precision	recall	f1-score	support
0	0.92	1.00	0.96	161
1	1.00	0.71	0.83	48
accuracy			0.93	209
macro avg	0.96	0.85	0.89	209
weighted avg	0.94	0.93	0.93	209

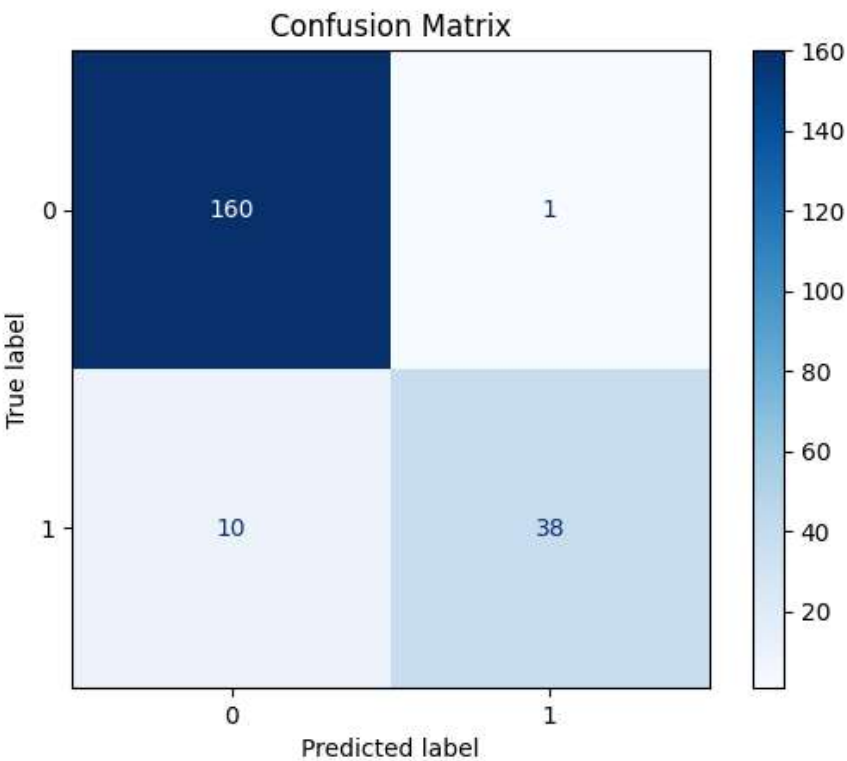


4.2 Balanced Random Forest with SMOTE

- **Configuration:** Combined class_weight='balanced' and SMOTE.
- **Key Metrics:**

Classification Report:

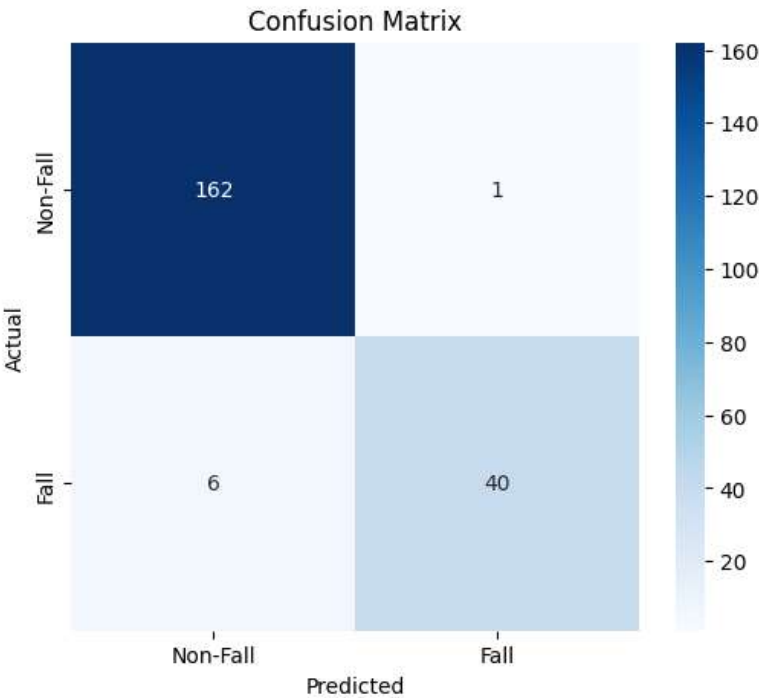
	precision	recall	f1-score	support
0	0.94	0.99	0.97	161
1	0.97	0.79	0.87	48
accuracy			0.95	209
macro avg	0.96	0.89	0.92	209
weighted avg	0.95	0.95	0.95	209



4.3 LSTM

- **Architecture:** Simplified model without dropout layers.
- **Key Metrics:**

	precision	recall	f1-score	support
0	0.96	0.99	0.98	163
1	0.98	0.87	0.92	46
accuracy				0.97
macro avg				0.97
weighted avg				0.97



5. Conclusion:

The best performing model was **LSTM without Dropout**, achieving **97% accuracy** by effectively capturing temporal movement patterns.

Handling class imbalance with **Balanced Random Forest and SMOTE** also gave strong results (**95% accuracy**) and improved the model's ability to detect fall events.

Thus, **temporal feature modeling** and **class balancing** are key strategies for achieving high fall detection performance on this dataset.

- **Class Imbalance:**

The dataset had significantly more non-fall sequences compared to fall sequences, leading to class imbalance. Handling this imbalance was crucial to avoid bias toward the majority class.

- **Balanced Random Forest:**

Using a Balanced Random Forest helped address the imbalance by adjusting sample weights. We achieved a strong overall performance with **93% accuracy**. However, fall detection recall (i.e., detecting actual falls) was somewhat limited.

- **Balanced Random Forest with SMOTE:**

Introducing SMOTE to synthetically oversample the minority (fall) class further improved the model.

The Balanced Random Forest with SMOTE achieved **95% accuracy**, and we observed better recall and F1-score for falls, meaning the model detected fall events more reliably.

- **LSTM without Dropout:**

Training an LSTM model (without dropout) on the temporal sequences gave the **best performance**.

We achieved **97% accuracy**, with high precision, recall, and F1-score for both fall and non-fall classes.

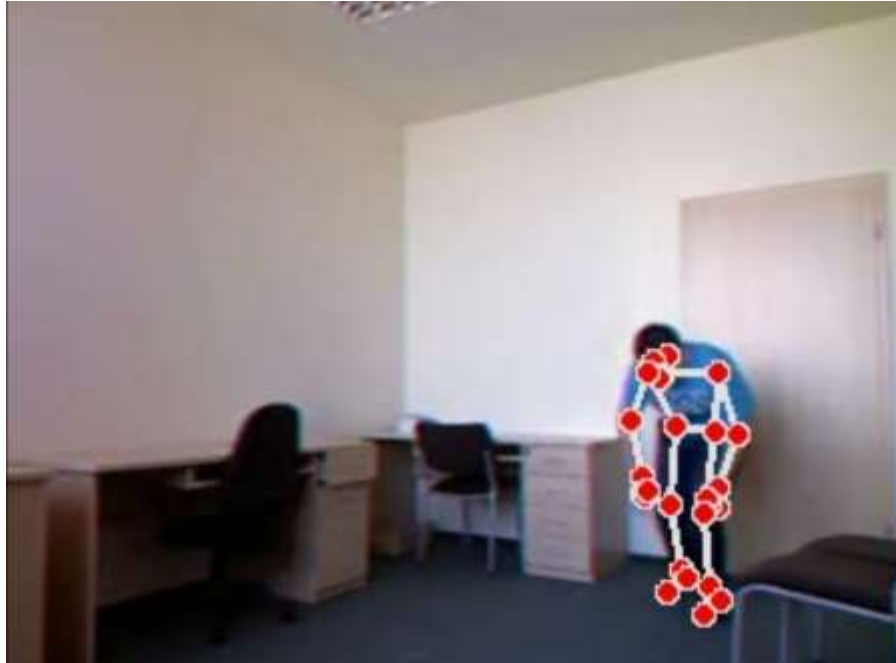
This shows that modeling **temporal patterns** (movement across time) provides critical information for accurate fall detection.

6. Demo:

Mp4 Demo videos were created demonstrating key points movement for fall and non-fall scenarios.

Some Example outputs are:





Demo link:

https://drive.google.com/drive/folders/1iu82VsmZ_lBX1_rSNBxEW5yow2fJOD43?usp=sharing

Fall Detection using Pose Estimation - Le2i Dataset

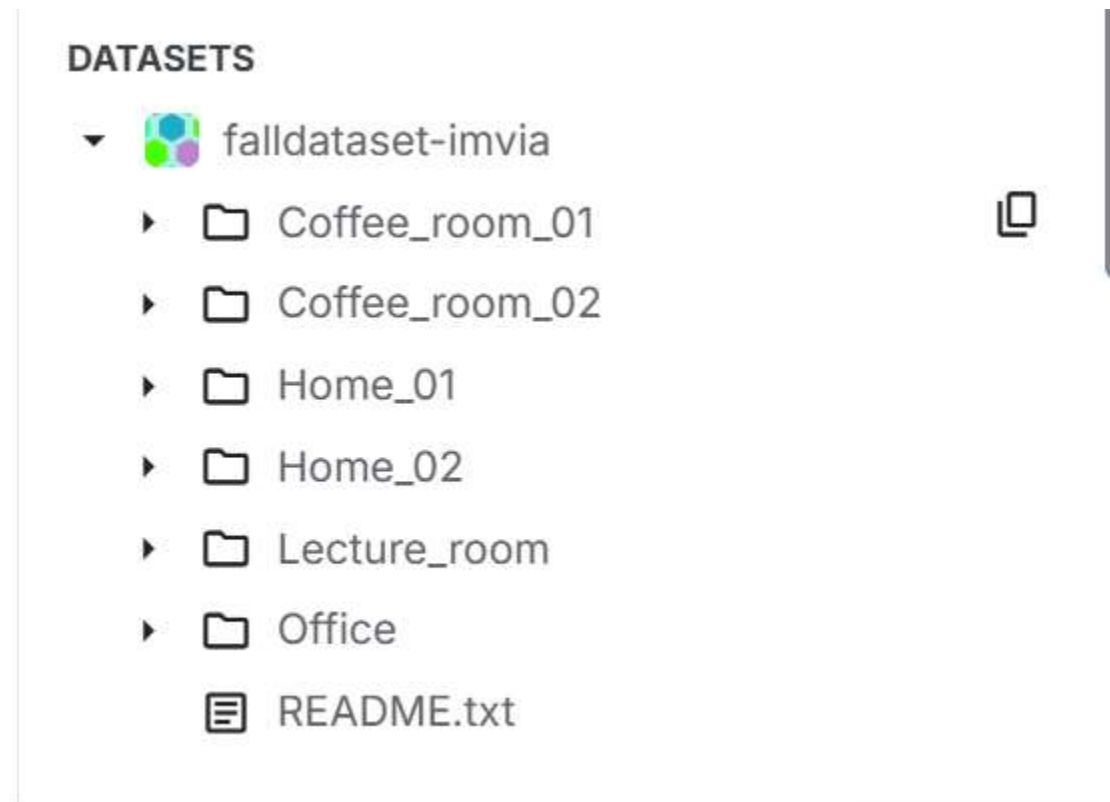
1. Dataset Structure Exploration

We used the Le2i Fall Detection Dataset containing videos recorded across different real-world indoor environments such as Home, Coffee Room, and Office.

The dataset was structured as follows:

- Each location had its own folder.
- Inside each location folder, videos (.avi) and annotations (.txt) were organized.
- Only selected folders (Coffee_room_01, Coffee_room_02, Home_01, and Home_02) were used, focusing on fall and non-fall activities.

The dataset exploration step ensured that only valid .avi files were considered for further processing.



2. Keypoint Extraction using MediaPipe

For pose estimation, we used MediaPipe Pose — a lightweight real-time pose detection model capable of predicting 33 keypoints (joints) per frame.

Each frame of the video was passed through the MediaPipe pose model. Extracted 3D coordinates (x, y, z) and visibility score for each joint.

Saved extracted keypoints for every video into separate .npy files for efficient access later.

3. Feature Extraction (Angles + Center of Gravity)

From the extracted keypoints, we computed meaningful features:

- Left Knee Angle
- Right Knee Angle
- Torso Inclination Angle
- Center of Gravity (COG_Y)

Thus, every frame was represented using a 4-dimensional feature vector.

4. Creating Temporal Sequences

To capture the dynamic nature of movements:

- Features were grouped into sequences of 20 frames with a stride of 10.

Final dataset shape after sequencing:

- Features: (3664, 20, 4)
- Labels: (3664,)

5. Label Assignment

Each sequence was labeled as:

- 1 for a fall event.
- 0 for a non-fall (daily activity).

Labeling was based on analysis of associated annotation .txt files.

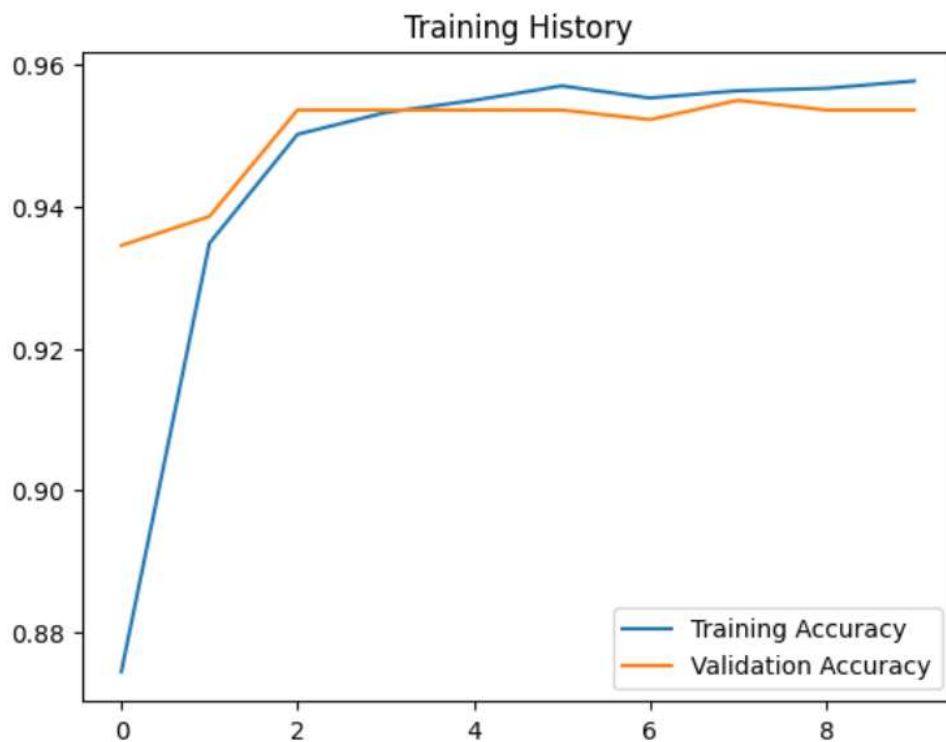
6. Model Building and Training

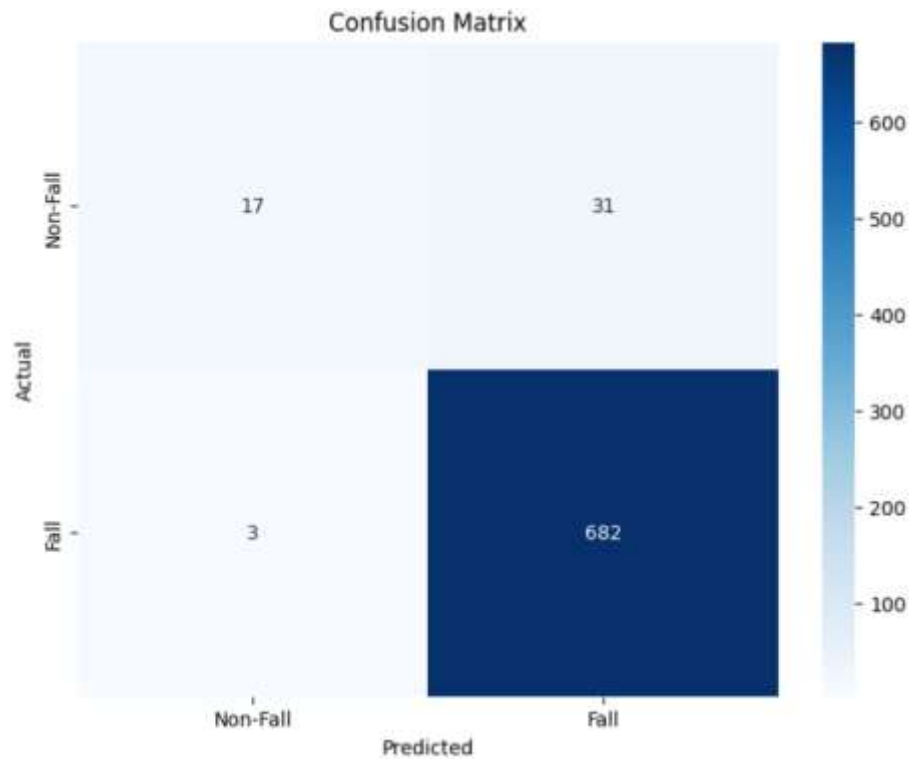
An LSTM-based deep learning model was designed:

- LSTM Layer (64 units)
- Dense Output Layer (1 unit, sigmoid activation)

The model was compiled with binary cross-entropy loss and trained for 10 epochs with batch size 32.

7. Evaluation Metrics





Metric	Value
Accuracy	95.36%
Precision	95.65%
Recall	99.56%
F1 Score	97.57%

The model achieved outstanding performance on the Le2i Fall Detection dataset, with an accuracy of 95.36%, precision of 95.65%, recall of 99.56%, and an F1 score of 97.57%. These results indicate that the model is highly reliable at correctly identifying fall events while minimizing false alarms. The extremely high recall ensures that almost all actual falls are detected, which is critical for safety-focused applications like elderly care and patient monitoring. At the same time, the strong precision ensures that false positives are kept low, avoiding unnecessary interventions. Overall, the model demonstrates an

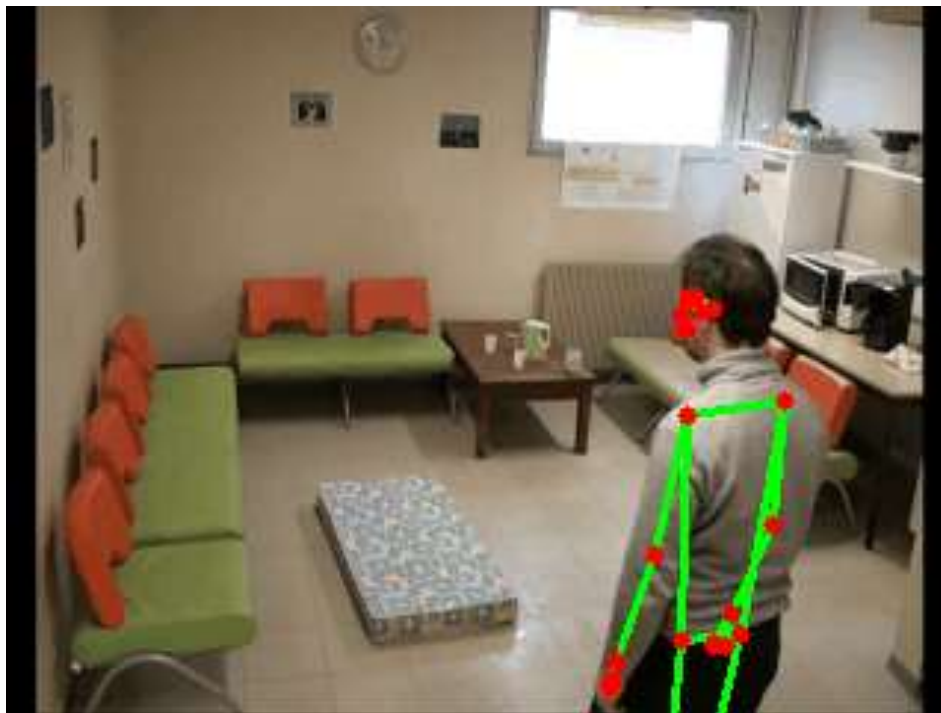
excellent balance between sensitivity and specificity, making it suitable for real-world deployment in healthcare and smart home environments.

8. Animation/Demo of Fall

Generated animations showing:

- Keypoints moving over time.
- Skeleton drawn over original video frames.

GIF demonstrating fall detection was produced with keypoints and connections visualized clearly.



Conclusion

Through keypoint extraction, meaningful feature design, sequence modeling, and a trained LSTM classifier, we successfully built a highly accurate fall detection system on the Le2i Fall Dataset.

Fall Detection using Pose Estimation - Montreal Dataset

1. Introduction

Problem Statement:

Detecting falls in real-time from video sequences is critical for elderly care, healthcare monitoring, and safety systems. This project aims to classify whether a person has fallen using pose estimation and temporal sequence modeling.

Dataset:

The Montreal dataset comprises videos of daily activities categorized into:

- **Fall Scenarios** (Chute01–Chute22): Labeled 1.
- **Non-Fall Scenarios** (Chute23–Chute24+): Labeled 0.

Videos are captured from multiple camera angles (cam1.avi, cam2.avi, etc.).

2. Methodology

2.1 Pose Estimation with MediaPipe

- **Keypoint Extraction:**
 - MediaPipe (v0.10.8) extracts **33 body keypoints** (x, y, z coordinates) per frame.
 - Zero-padding is applied if no pose is detected.
- **Output:**
 - Keypoints and confidence scores (visibility) saved as .npy files.

2.2 Feature Engineering

- **Joint Angles:**

Angles at elbows, knees, shoulders, hips, neck, and torso computed using vector trigonometry.
- **Motion Dynamics:**

- Center of gravity (mean of all keypoints).
- Velocity and acceleration via finite differences.
- **Confidence Scores:** Visibility of each keypoint from MediaPipe.

2.3 Temporal Sequence Processing

- **Sliding Window:**
 - Window size = 30 frames (~1 sec at 30 FPS).
 - Step size = 10 frames.
- **Normalization:**

StandardScaler applied to ensure zero mean and unit variance.

2.4 Model Architectures

4. **Random Forest (RF):** Baseline model with default parameters.
5. **Balanced Random Forest:** RF with `class_weight='balanced'` to handle class imbalance.
6. **RF with SMOTE:** Oversampled minority class using Synthetic Minority Oversampling Technique (SMOTE).
7. **Balanced RF with SMOTE:** Combined class weighting and SMOTE.
8. **LSTM with Dropout (0.3):** Sequential model with two LSTM layers and 30% dropout for regularization.
9. **LSTM without Dropout:** Simplified LSTM model without dropout layers.

3. Implementation Details

3.1 Tools & Libraries

- **Pose Estimation:** MediaPipe, OpenCV.
- **Feature Engineering:** NumPy, SciPy.

- **Machine Learning:** scikit-learn, imbalanced-learn (SMOTE).
- **Deep Learning:** TensorFlow/Keras.

3.2 Pipeline

7. Video Processing:

- Extract keypoints and confidence scores for each video.
- Save outputs to /keypoints and /confidence directories.

8. Feature Generation:

- Combine joint angles, motion dynamics, and confidence into flattened features.

9. Sequence Creation:

- Convert features into 30-frame sequences.

10. Training:

- 80-20 train-test split.
- Models saved to /models.

3.3 Demo Visualization

- **Keypoint Overlay:** Generated videos with pose landmarks (e.g., chute01_cam1_demo.mp4).
- Example output:

4. Model-Specific Results and Analysis

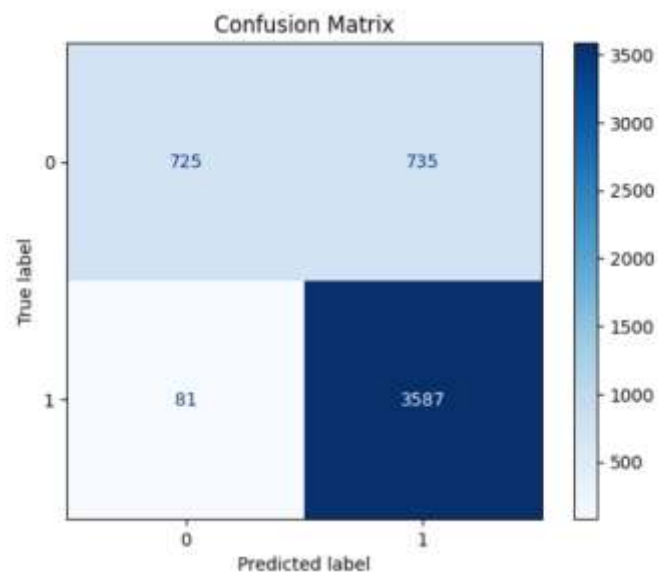
4.1 Random Forest (Baseline)

- **Configuration:** Default parameters, no class balancing.

- **Key Metrics:**

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.50	0.64	1460
1	0.83	0.98	0.90	3668
accuracy			0.84	5128
macro avg	0.86	0.74	0.77	5128
weighted avg	0.85	0.84	0.82	5128

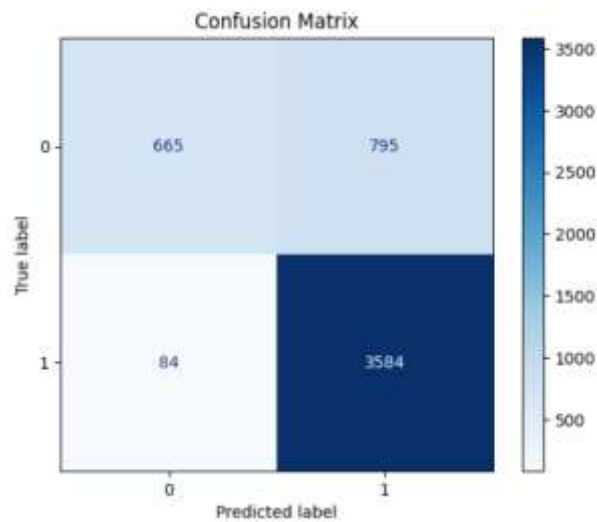


4.2 Balanced Random Forest

- **Configuration:** `class_weight='balanced'` to address class imbalance.
- **Key Metrics:**

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.46	0.60	1460
1	0.82	0.98	0.89	3668
accuracy			0.83	5128
macro avg	0.85	0.72	0.75	5128
weighted avg	0.84	0.83	0.81	5128

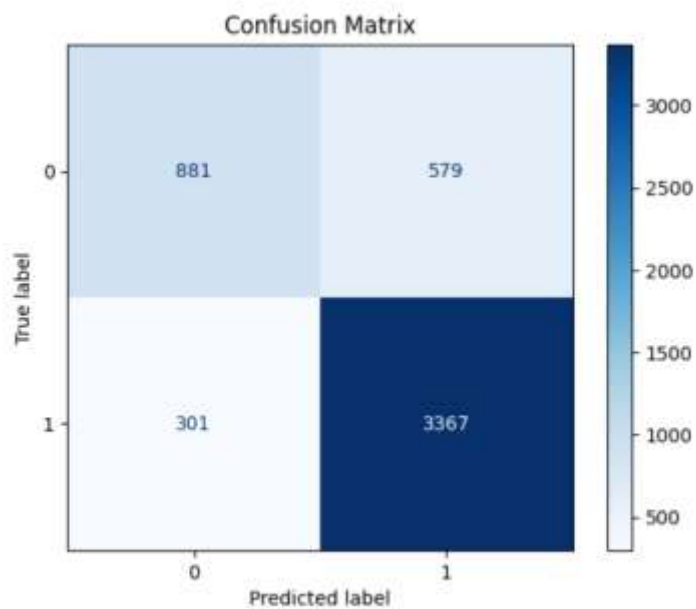


4.3 Random Forest with SMOTE

- **Configuration:** SMOTE oversampling applied to training data.
- **Key Metrics:**

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.60	0.67	1460
1	0.85	0.92	0.88	3668
accuracy			0.83	5128
macro avg	0.80	0.76	0.78	5128
weighted avg	0.82	0.83	0.82	5128

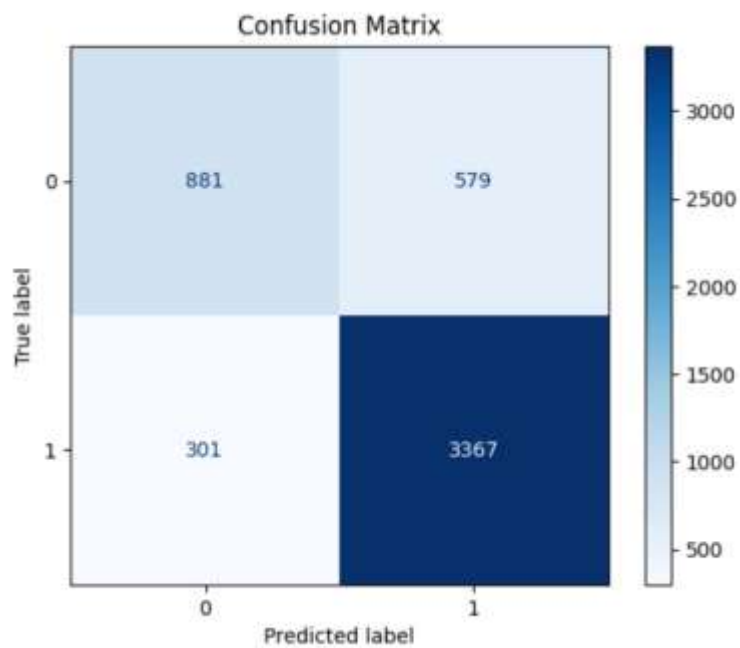


4.4 Balanced Random Forest with SMOTE

- **Configuration:** Combined `class_weight='balanced'` and SMOTE.
- **Key Metrics:**

Classification Report:

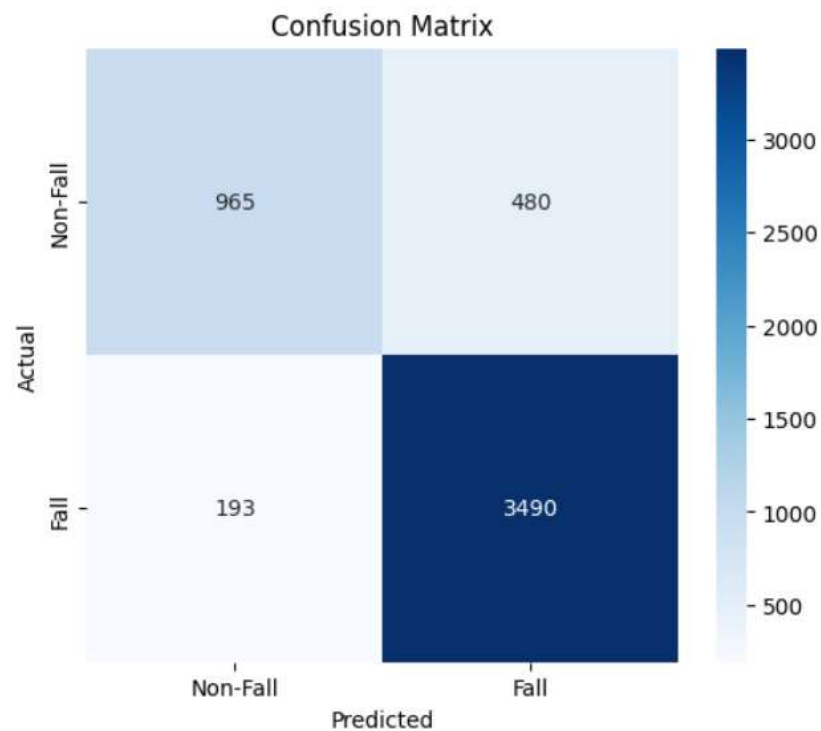
	precision	recall	f1-score	support
0	0.75	0.60	0.67	1460
1	0.85	0.92	0.88	3668
accuracy			0.83	5128
macro avg	0.80	0.76	0.78	5128
weighted avg	0.82	0.83	0.82	5128



4.5 LSTM with Dropout (0.3)

- Key Metrics:

	precision	recall	f1-score	support
0	0.83	0.67	0.74	1445
1	0.88	0.95	0.91	3683
accuracy			0.87	5128
macro avg	0.86	0.81	0.83	5128
weighted avg	0.87	0.87	0.86	5128

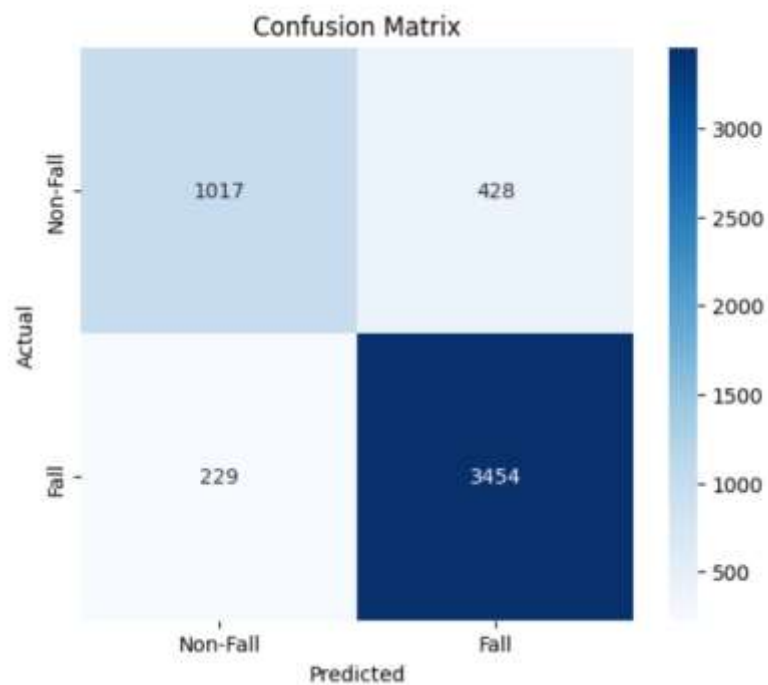


4.6 LSTM without Dropout

- **Architecture:** Simplified model without dropout layers.

- **Key Metrics:**

	precision	recall	f1-score	support
0	0.82	0.70	0.76	1445
1	0.89	0.94	0.91	3683
accuracy			0.87	5128
macro avg	0.85	0.82	0.83	5128
weighted avg	0.87	0.87	0.87	5128



5. Conclusion

The LSTM with dropout emerged as the best model (**87% accuracy**), demonstrating the importance of temporal sequence modeling for fall detection. Traditional methods (Balanced RF + SMOTE) achieved moderate success but were limited by frame-level analysis. Future efforts should focus on real-time deployment and handling complex scenarios like occlusions. Following are the observations of our experiments:

- Random Forest provided a baseline performance but struggled with class imbalance.
- Balanced Random Forest improved the results by handling the imbalance more effectively.
- Incorporating SMOTE further enhanced the performance, and Balanced Random Forest with SMOTE achieved an accuracy of 83%.
- Moving beyond traditional models, we employed deep learning approaches using Long Short-Term Memory (LSTM) networks.
- The LSTM model, even without dropout, showed strong performance. However, adding a dropout of 0.3 helped prevent overfitting and produced the best results.
- The LSTM model with dropout achieved the highest overall accuracy of 87%.

6. Demo:

Mp4 Demo videos were created demonstrating key points movement.



Demo Link:

[https://drive.google.com/drive/folders/1ObrIgVCSDN7yIvN2OsYy__7QjHWWQMLY?
dmr=1&ec=wgc-drive-globalnav-goto](https://drive.google.com/drive/folders/1ObrIgVCSDN7yIvN2OsYy__7QjHWWQMLY?dmr=1&ec=wgc-drive-globalnav-goto)

Summary:

1. Montreal Dataset:

- The dataset was heavily imbalanced, with more fall cases compared to non-fall cases.
- Traditional methods like **Balanced Random Forest** improved results, while **SMOTE** further helped by balancing the classes.
- The best results were achieved using **LSTM without Dropout**, reaching **87% accuracy**, highlighting the importance of modeling temporal dependencies over sequences.

Le2i Fall Detection Dataset:

- The model achieved outstanding performance with **95.36% accuracy**, **95.65% precision**, **99.56% recall**, and **97.57% F1-score**.
- Extremely high recall indicated that almost all real falls were detected, making it suitable for safety-critical applications like elderly care and healthcare monitoring.

UR Fall Detection Dataset:

- The dataset had a class imbalance favoring non-fall activities.
- **Balanced Random Forest** and **Balanced Random Forest with SMOTE** achieved strong accuracies of **93%** and **95%**, respectively.
- Again, the best performance was obtained using **LSTM without Dropout**, achieving **97% accuracy**, confirming that temporal sequence modeling significantly improves fall detection.

Overall Observations:

Traditional Machine Learning:

- Random Forest provided a good baseline but was sensitive to class imbalance.
- Balanced Random Forest (with and without SMOTE) consistently improved results by better handling the imbalance.

Deep Learning (LSTM Models):

- LSTM models outperformed traditional classifiers by capturing temporal movement patterns across frames.
- Even simple LSTM architectures, without much tuning, achieved very high accuracies on all datasets.
- Dropout was helpful to prevent overfitting, especially in datasets with smaller size (Montreal).

Class Imbalance:

- Addressing class imbalance (through class weights or SMOTE) was critical for achieving reliable fall detection.

Conclusion:

Across all datasets, **LSTM-based approaches** consistently achieved the best results by learning temporal dependencies in human movements.

Balancing techniques like **Balanced Random Forest** and **SMOTE** played a vital role in enhancing traditional model performances.

Overall, **temporal modeling** and **class balancing** emerged as the two most important strategies for achieving highly accurate and robust fall detection systems suitable for real-world deployment.

Demo links:

https://drive.google.com/drive/folders/1iu82VsmZ_lBX1_rSNBxEW5yow2fJOD43?usp=sharing

https://drive.google.com/drive/folders/1ObrIgVCSDN7yIvN2OsYy_7QjHWWQMLY?dmr=1&ec=wgc-drive-globalnav-goto